

Part 4 — Critical Analysis of Clinical Risks

Project: Pharmacy pipeline — medication data, note extraction, natural-language querying, and a perioperative medication-check module. **Data:** Fully synthetic (no real PHI). Guideline content transcribed from KHCC *Preoperative Medication Management Guidelines* (GDLPT-25). **Framing:** This is a prototype. The analysis below is written from the assumption that the system would sit *beside* a clinician as decision support, never in front of one as an autonomous decision-maker. Every risk is judged against that standard.

1. Risks in information extraction from clinical notes (Part 2)

The extraction step uses an LLM with a Pydantic schema to pull drug interactions out of free-text notes. It worked well on my 12 test notes, but the failure modes it exposed are the ones that matter clinically.

False negatives are the dangerous failure, not false positives. When the model finds no interaction, it returns an empty list — which is indistinguishable, to the reader, from "this patient has been checked and is safe." A missed warfarin–NSAID pairing does not announce itself; it simply produces silence. In a medication-safety tool, silence must never be interpreted as an all-clear. Any deployment would need to treat "none found" as *not confirmed* rather than *confirmed absent*, and to surface a confidence signal so a clinician knows how much weight the result carries.

The model bends the schema. In note 4, the system reported "bevacizumab + surgery" as a drug–drug interaction. Clinically the underlying fact is correct — bevacizumab impairs wound healing — but "surgery" is not a drug, and this is a perioperative flag, not an interaction. The model stretched my definition to fit what it saw. This shows that a schema constrains *format* but not *clinical correctness*: structured output guarantees the shape of the data, not its validity. Downstream validation (e.g. checking that both members of a pair are real drugs in the formulary) is required before any extracted interaction is trusted.

Output is non-deterministic. Running the same 12 notes on different occasions produced 8 interactions once and 11 another time — the model intermittently found an additional fluoxetine–apixaban pair. For an auditable clinical system this is a genuine problem: the same input must not yield different safety conclusions on different runs. Mitigations include lowering sampling temperature, running extraction more than once and reconciling, and — most importantly — never presenting extraction as a single authoritative answer without a human check.

Evidence and auditability. Capturing an `evidence_span` (the exact phrase from the note that justifies each extraction) is a deliberate safety feature: it lets a pharmacist verify *why* the system flagged something, rather than trusting an opaque output.

2. Risks in natural-language querying (Part 3)

The text-to-SQL interface lets a pharmacist ask questions in plain English. Its risks are subtle because, unlike extraction, it usually returns *something*.

Silent wrong answers are worse than errors. A query that crashes is obvious and harmless. A query that runs but quietly omits a filter — for example, returning all warfarin orders including discontinued ones because the model forgot `order_status = 'active'` — returns a plausible, confident, *wrong* answer.

The pharmacist has no signal that anything is off and will act on it. This is the core danger of the interface: it converts a language model's uncertainty into what looks like a definitive database result.

Mitigation — the trust boundary. The system never lets the model's SQL touch the data unchecked. Two independent layers stand between the LLM and the database:

1. A guard (`is_safe`) that rejects anything which is not a single read-only `SELECT` — blocking `INSERT` , `UPDATE` , `DELETE` , `DROP` , and multi-statement injections such as `SELECT 1; DROP TABLE patients .`
2. A read-only database connection, so that even if a harmful query slipped past the guard, SQLite physically refuses the write.

This is defence in depth: the query is validated *and* the channel is incapable of causing harm. It reflects the general principle that an LLM's output should be treated as an untrusted suggestion, not a trusted command.

Residual risk. These layers protect *data integrity* (nothing can be altered), but not *answer correctness* (a read-only query can still be clinically wrong). The interface is therefore appropriate for exploration and drafting, but its answers should be treated as a starting point a clinician verifies, not a source of truth.

3. Risks in the perioperative decision-support module (stretch)

This module joins a patient's active orders to the KHCC guideline rules and flags medications that must be held or stopped, with a calculated stop-by date. It is the most clinically consequential component, and it carries the most instructive risks.

Name-matching misses drugs in the same class — the most serious gap. The current version matches a patient's drug to a guideline rule by drug name. Because GDLPT-25 lists ACE inhibitors by specific examples (captopril, enalapril, candesartan, valsartan), a patient on **lisinopril** is not matched and receives no flag, even though the guideline clearly intends to cover their drug. For a benign class this is a nuisance; for anticoagulants or antiplatelets it would be a patient-safety hole, because the failure is once again silent. The correct fix is to match on **drug class** rather than name, so that any member of a covered class fires the flag whether or not the guideline names it explicitly. This is the single most important improvement the module needs before it could be trusted.

Free-text timing does not map cleanly to arithmetic. The stop-by date is computed by parsing the guideline's timing text. For an entry like "at least 3 days before (ibuprofen up to 24 h before elective)," my parser took the conservative "3 days," producing an earlier stop date than a specific case might require. Erring conservative (stopping earlier) is the safer direction for a bleeding-risk drug, but it illustrates that human-written guidelines are not written for machine arithmetic, and that any date the system produces is an approximation a clinician must confirm against the actual procedure.

Decision support, not decision. By design, the module flags, states the guideline recommendation, calculates a provisional date, and routes to the appropriate service (cardiology for anticoagulation bridging, oncology for VEGF inhibitors and SERMs). It never instructs an automatic hold. This mirrors the guideline itself, which repeatedly defers bridging and borderline decisions to specialist judgement. The system's role is to make sure the right question reaches the right person in time — not to answer it.

The chemotherapy-nadir concept follows the same safe pattern. Where the system reads a chemotherapy administration date from a nursing note and calculates days since treatment, the *reading* is done by the LLM

and the *date arithmetic* is done by deterministic code. Language models are unreliable at arithmetic; separating "extract the date as written" from "compute the interval" keeps the trustworthy part auditable.

4. Cross-cutting risks

Guideline lifecycle and staleness. The source guideline (GDLPT-25) carries a revision-due date of August 2023. A clinical rules engine is only as safe as the guideline it encodes, so the rules table stores `guideline_source` and `version_date` on every rule. A production system must display which guideline version it is citing and flag when that version is overdue for review — otherwise it risks giving confidently outdated advice.

Protected health information. The entire pipeline runs on synthetic data. Real consultation notes contain PHI and cannot be sent to an external LLM API as-is; a real deployment would require an approved, in-institution model or de-identification before any text leaves the hospital boundary. Secret handling in this prototype already follows that discipline — the API key is stored outside the code in an environment file and excluded from version control.

Human-in-the-loop as the governing principle. Every risk above resolves to the same mitigation: the clinician stays in the loop. Extraction is verified, not trusted; query answers are checked, not acted on blindly; flags route to a decision-maker rather than executing a decision. The system's value is in surfacing and organising information a busy surgeon might miss, not in replacing clinical judgement.

5. Summary

Risk	Where	Why it matters	Mitigation
False-negative extractions	Part 2	silence looks like safety	treat "none" as unconfirmed; surface confidence
Schema-bending / invalid pairs	Part 2	valid format $\neq$ valid clinical content	validate against formulary
Non-deterministic output	Part 2	same input, different safety answer	low temperature; reconcile runs; human check
Silent wrong query results	Part 3	plausible but incorrect, trusted	verification by clinician; treat as draft
Data alteration via query	Part 3	injection / accidental writes	SELECT-only guard + read-only connection
Class members missed (lisinopril)	Stretch	silent gap for un-named drugs	match on drug class , not name
Timing parsing imprecision	Stretch	guidelines aren't machine arithmetic	conservative default; clinician confirms date
Guideline staleness	System	outdated advice given confidently	version metadata; overdue-review flag
PHI exposure	System	real notes can't leave the institution	synthetic data; in-house model / de-identification

Conclusion. In its current form the system is a credible prototype for *decision support on synthetic data*. It is safe to use for exploration, drafting, and surfacing medications a clinician should review. It is **not** safe to use as an autonomous authority: its extractions can miss, its queries can mislead, and its class-matching has a known gap. The path to clinical readiness runs through class-based matching, output validation, guideline-version governance, an in-institution model for PHI, and — underpinning all of it — a clinician who remains the decision-maker.